Coding and Testing, part-II

S. S Satapathy

Data Flow Coverage Criteria

Control flow graph can be utilized to **track where** the **values** of the **variables** are **defined** and **where they** are **used** in the program. This helps to **indentify any unusual** scenario in **data usage** that might lead to possibility of **any bug** in the program.

S. S Satapathy

Data flow-based testing

```
int prime(int n){
        int i=2;
        while (i<n){
                if (n%i==0) break;
                i=i+1;
        }
         if(i==n) printf("Prime")
         else printf("Not Prime");
         return 0;
}
```

- In any program **data values** are created **(defined)** in some point (e.g. in the statement *int i=2;* i is defined) and **used** latter (in the condition *i<n*). This creation & use of data elements might repeat several times.
- Graph model of a program can be used to test how the values of the variables are changing during course of program execution. In this approach, **coverage criteria is defined** based on the **definition and use of the values (**of the variables).

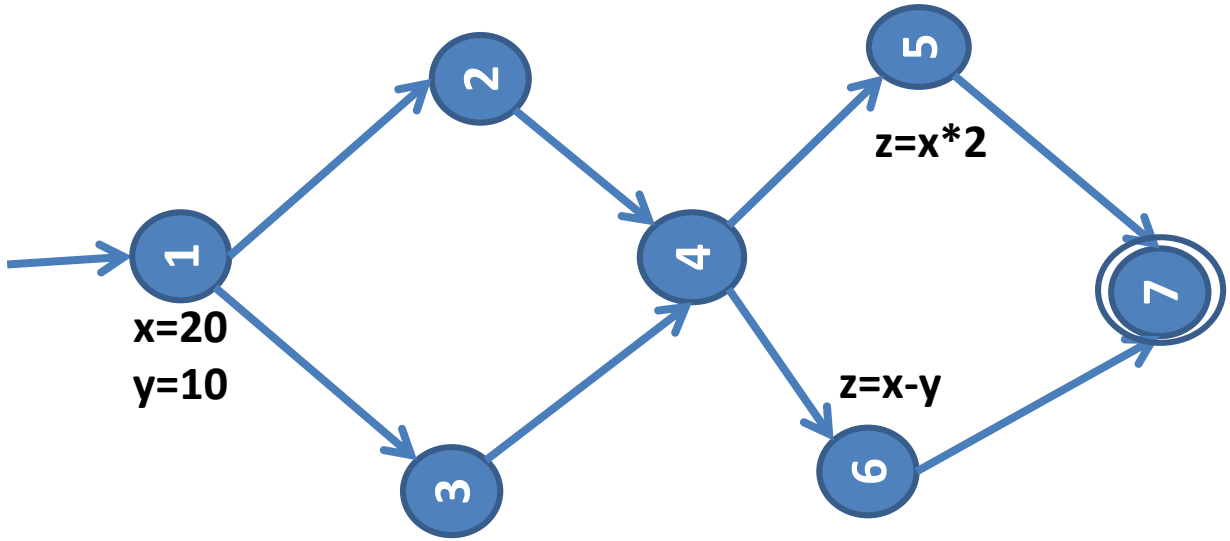
Definition and Use of Values

- A **definition (def)** **is a location where a value of a variable is stored into memory** (e.g. input, assignment etc..).
- A **use** **is a location where a value of a variable is accessed**.
- **Values** are **carried** from their **defs** to **uses**, called as **du-pairs** (**def-use pairs**).
- A **du-pair** **is a pair of locations** **(l_i, l_j)** such that a **variable v** is **defined at l_i** and **used at l_j**.
- **Data flow-based testing method selects test paths** of a program according to **the locations of the definitions and uses (du-pairs) of different variables** in a program.

Definition and Use of Values

- As a program can be represented in terms of a graphs, the **du-pair** **is a pair of nodes representing a** **path** **from one node to other**.
- Let the **set V** **represents** **all variables** of a **program modeled as a graph**.
- **The** **subset of V** **that each** **node n/edge e** **defines is called** **def(n)**/**def(e)**.
  - Graphs representing programs will **not have defs on edges**
- The **subset of V that each** **node n** /**edge e** **uses is called** **use(n)**/**use(e)**.

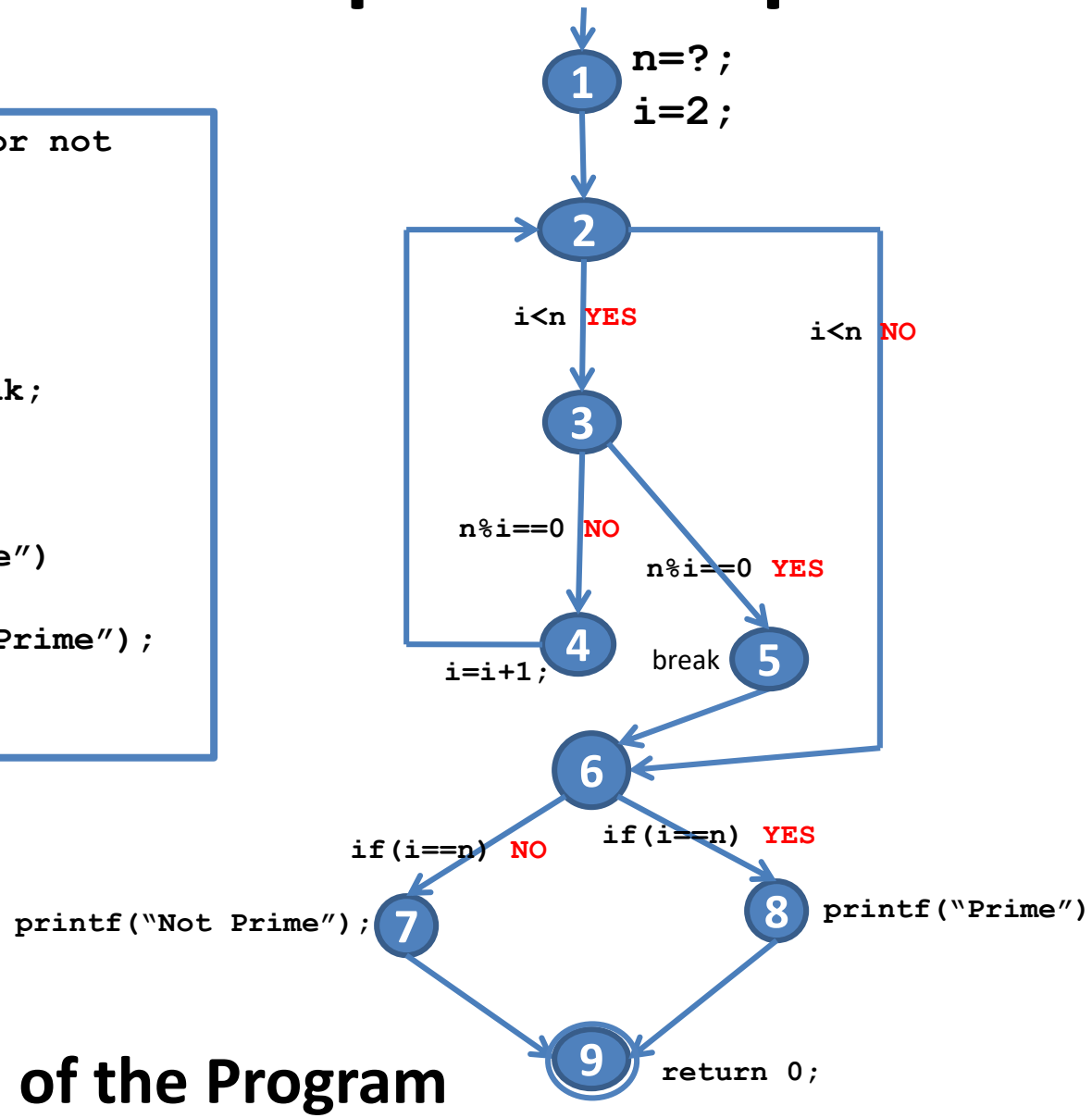
Def and Use set in a Graph: Example



- **Def sets of the nodes**
  - **def(1) = {x,y} in node 1, new values are assigned to x and y**
  - **def(5)={z} in node 5, new value is assigned to z**
  - **def(6)={z} in node 6, new value is assigned to z**
- **uses:**
  - **use(5) = {x} in node 5, value of x is used**
  - **use(6) = {x,y} in node 6, value of x and y are used**
- **It is a hypothetical example, all nodes/edges are not labeled. An actual example is given in the next slide**

Def and Use in Graphs: Example

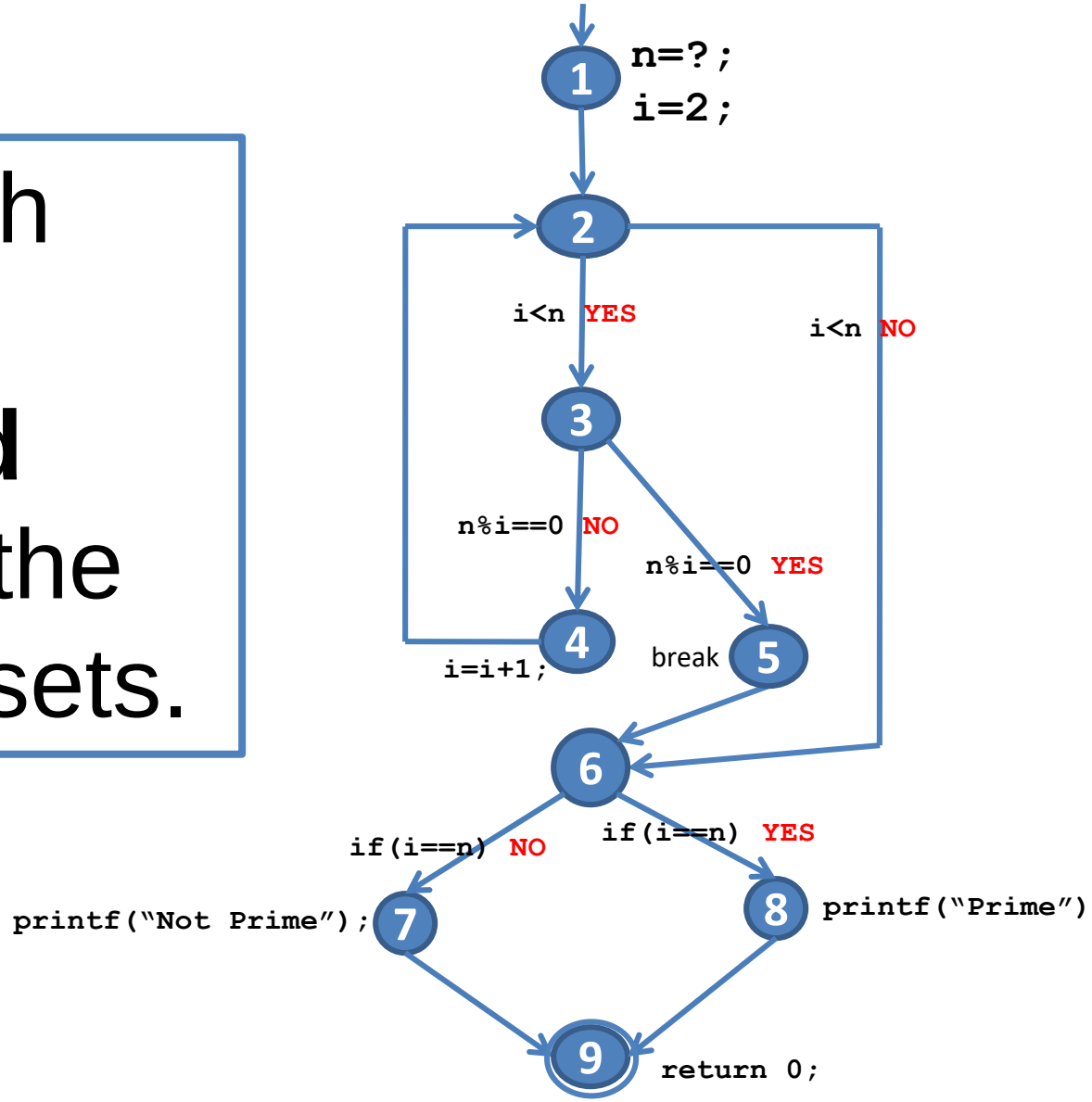
```
//Check if a number is prime or not
int prime(int n)
{
        int i=2;
        while (i<n){
                if (n%i==0)
                        break;
                i=i+1;
        }
         if(i==n)
                printf("Prime")
         else
                printf("Not Prime");
         return 0;
}
```



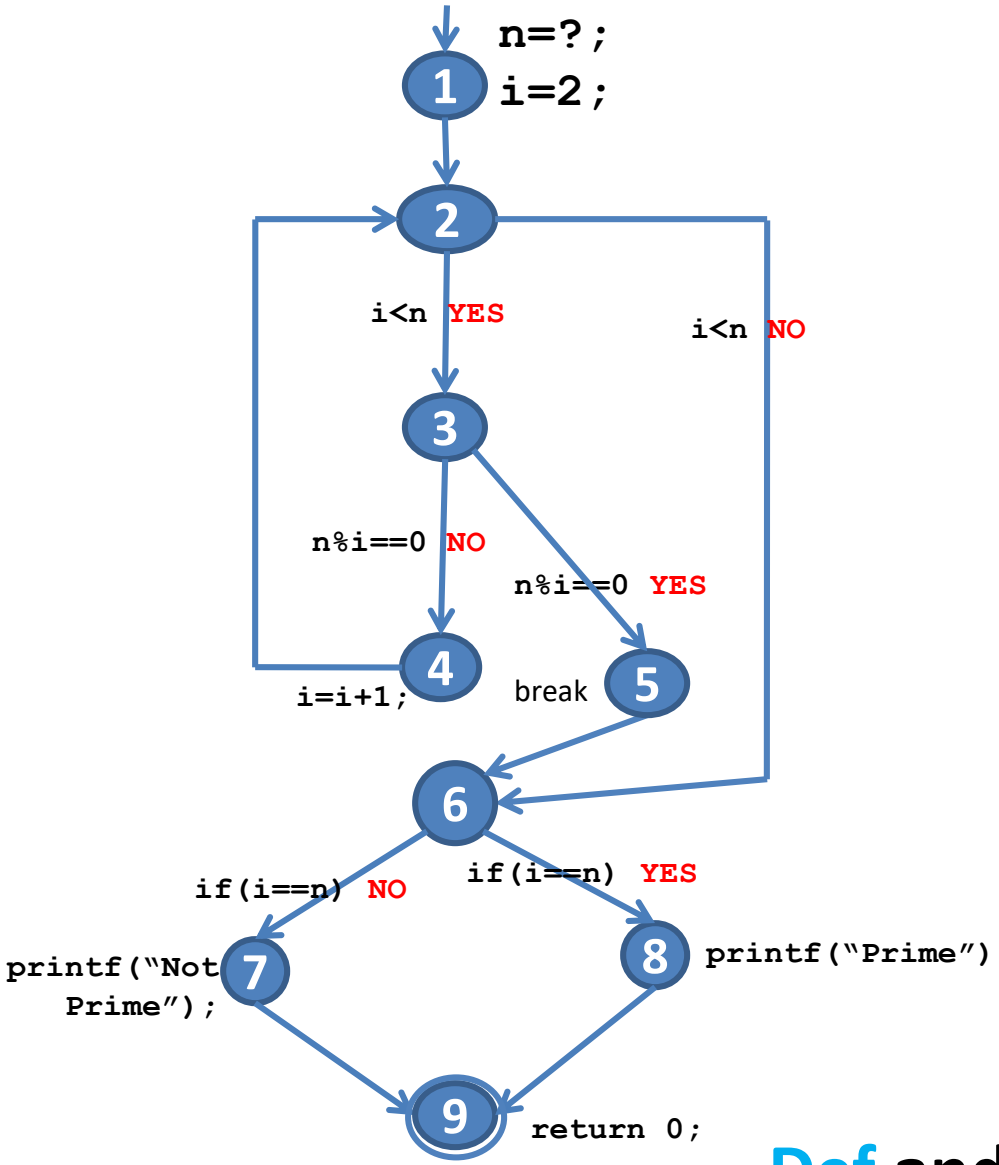
CFG of the Program

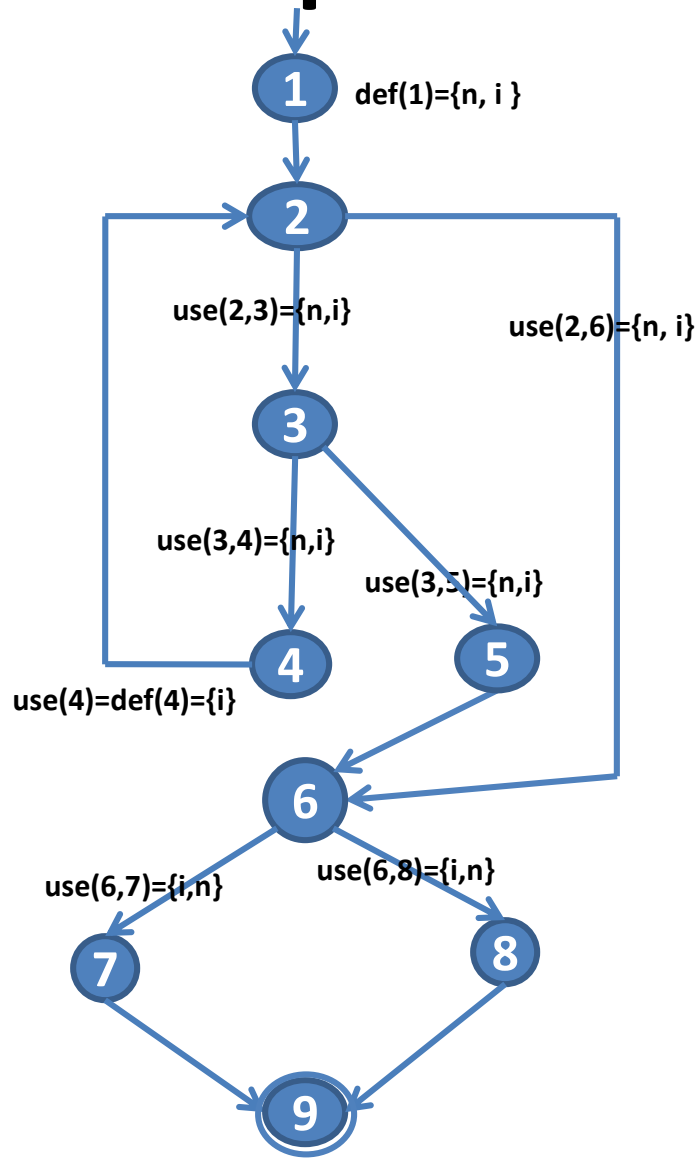
Def and Use in Graphs: Example

Find out which variable is **defined/used** where to get the **def** and **use** sets.



Def and Use in Graphs: Example

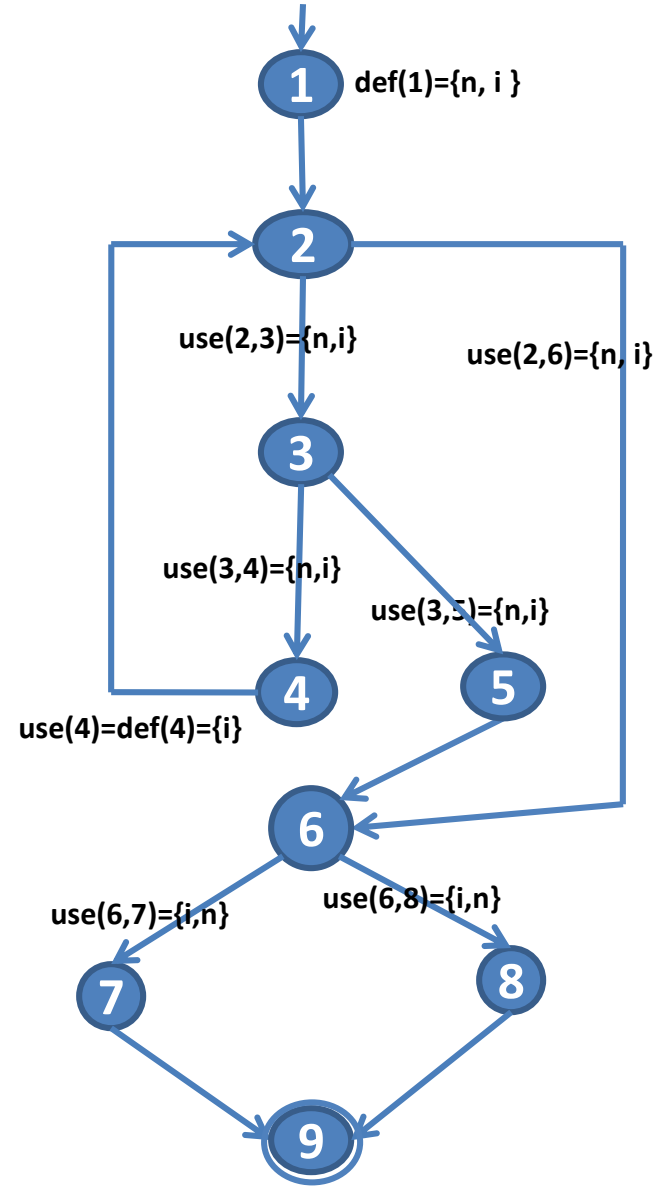




Def and use sets for the nodes/edges.

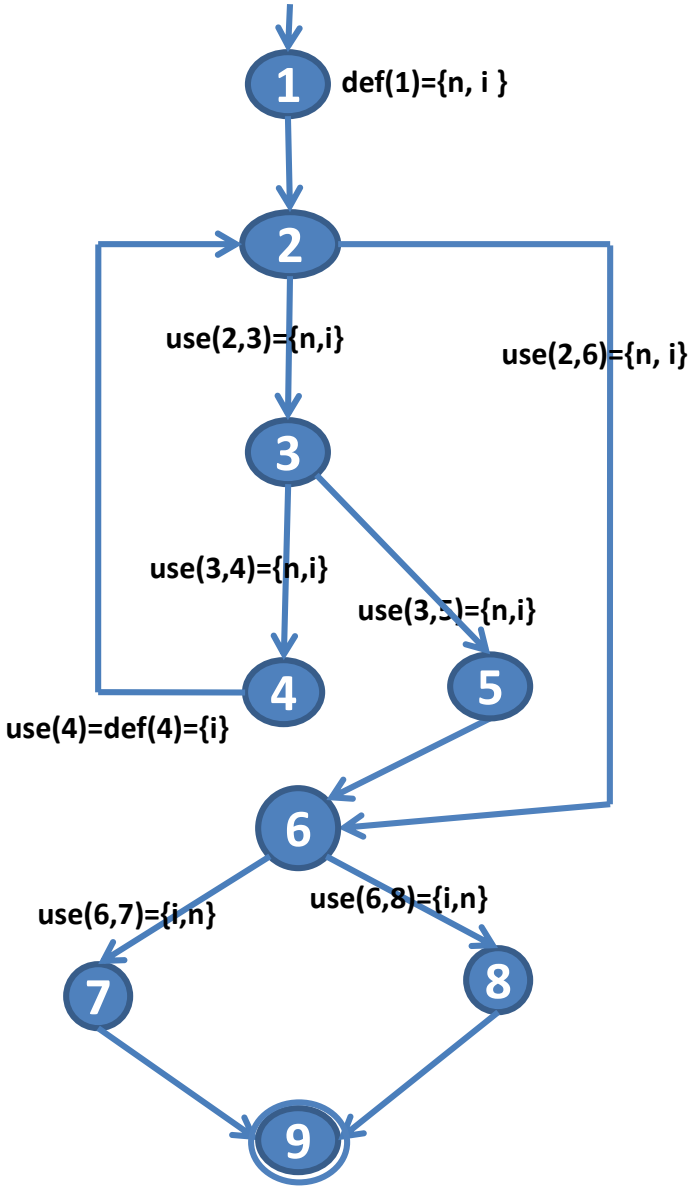
Some more information on Def and Use

- A **def** of a variable **may or may not reach** a particular **use**.
  - **A def** of a variable v at location l_i **will not reach use** of v at location l_j if there **is no path** from l_i to l_j.
  - The **value of v could be changes** by **another def** before it reaches an use.
  - For example, variable i is defined in node 1. This definition of i will not reach use of i in edge (3,5) in the path (1,2,3,4,2,3,5) because of one more def of i in node 4. However, def of i from node 1 will reach use of i in edge (3,5) through path (1,2,3,5).



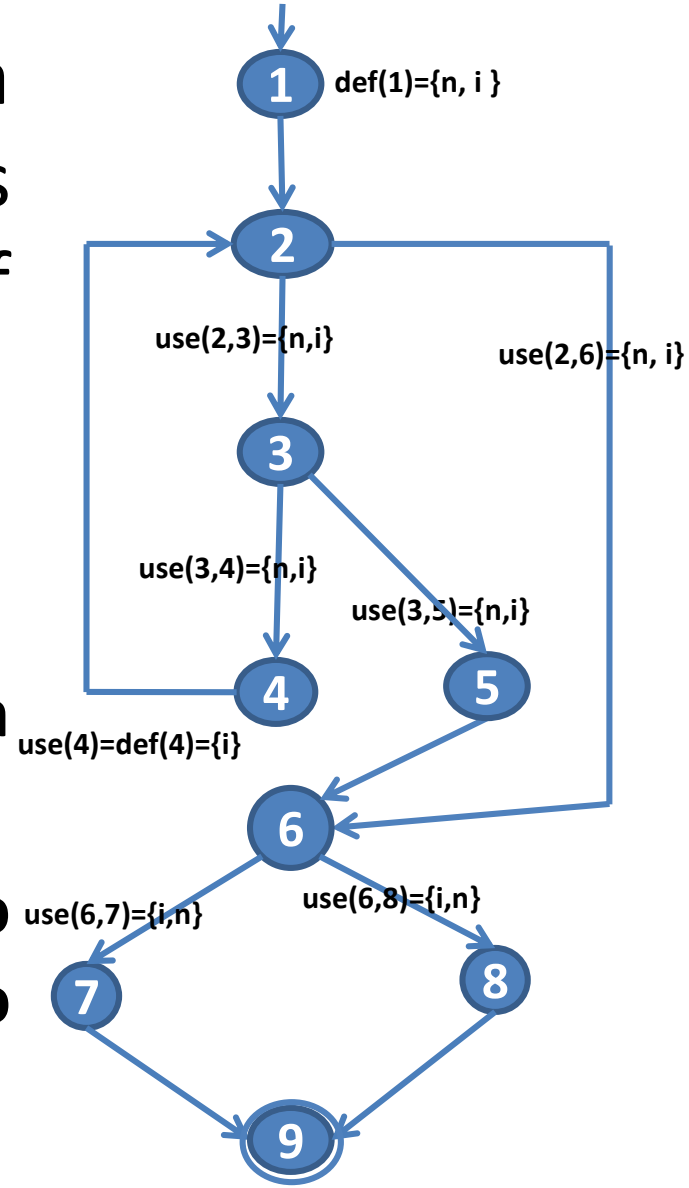
Some more information on Def and Use

- A **path from l_i to l_j** is **def-clear** with respect to variable v if **v is not given another value on any of the nodes or edges in the path**
  - **e.g: path (1,2,6) is def-clear** for variable **i**.
  - **Path (1,2,3,4,2,6) is not def-clear** for variable **i**.
- **If there is a def-clear path** from l_i to l_j, w.r.t. v, **the def of v at l_i reaches the use at l_j**.



Def-Use Paths or du-Path

- A **def-use path (du-path)** w.r.t. a **variable v** is a **simple path** that is **def-clear** from a **def** of **v** to **use** of **v.**
  - **du-paths** are parameterized by v.
  - They need to be **simple paths**
  - There may be **intervening uses** on the path
  - E.g: there is **du-path from node 1 to edge (6,7) and also from node 1 to edge (6,8)**

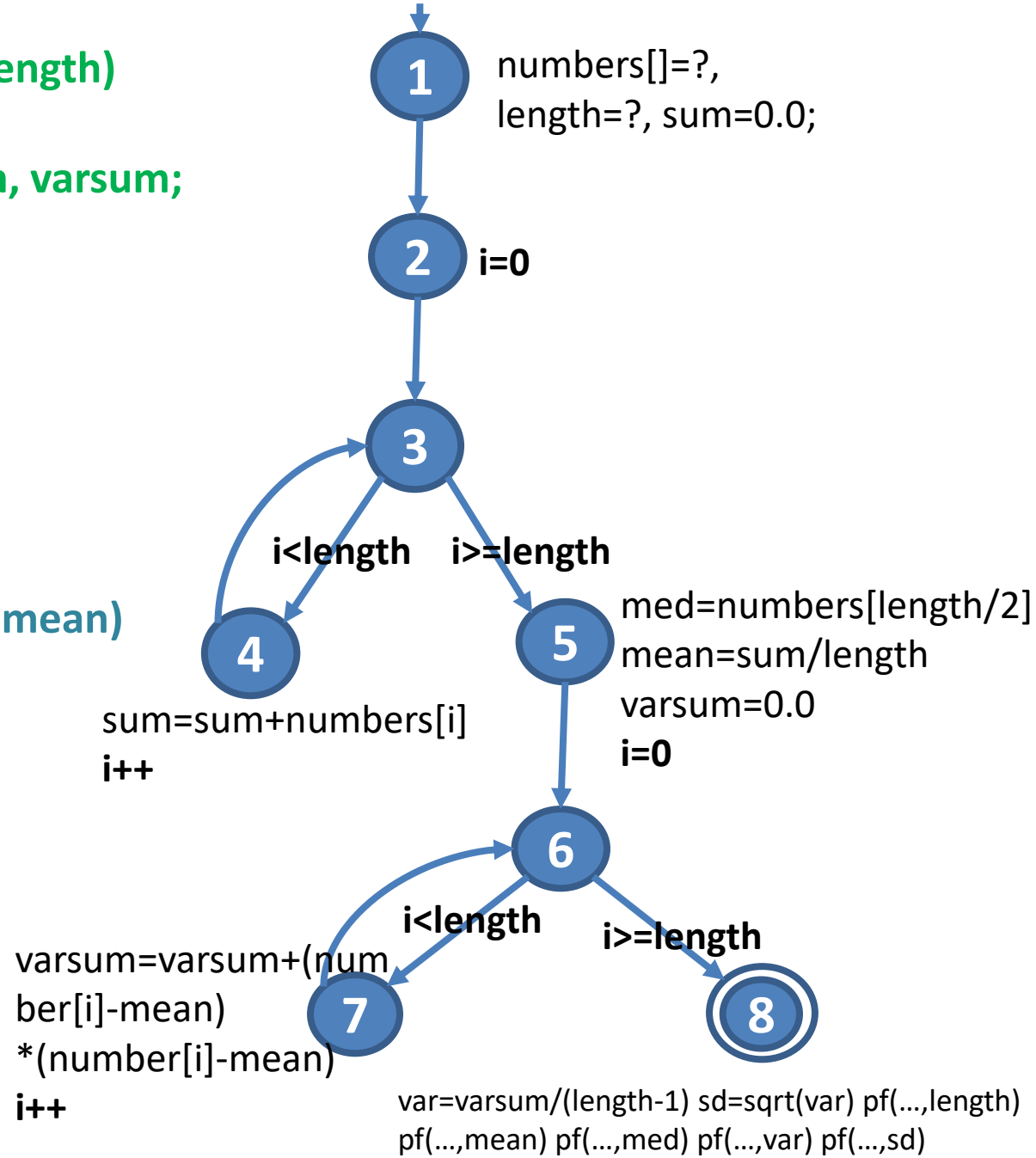


Def-Use Paths

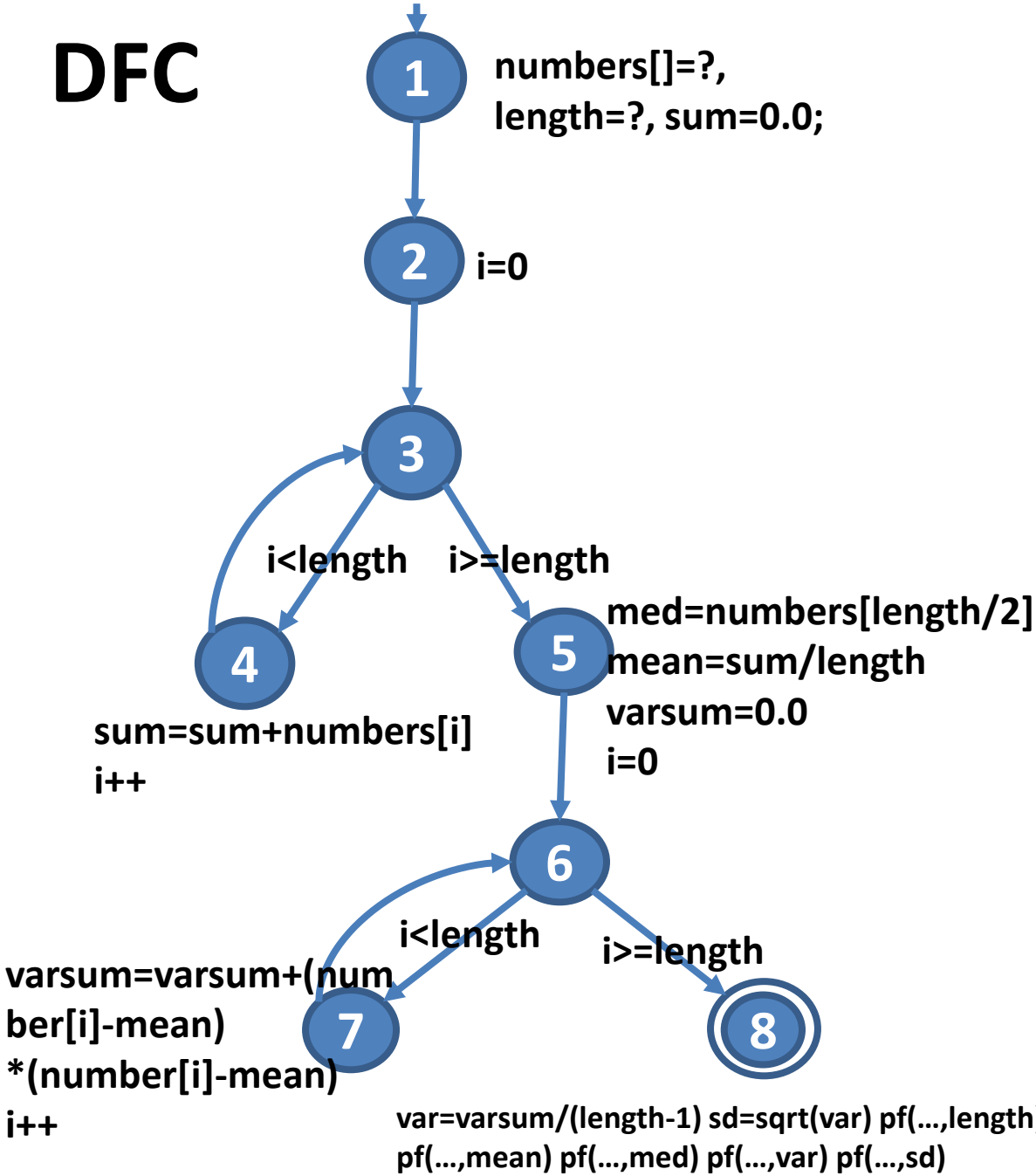
- A **def-use path (du-path)** analysis is useful for detecting several defects in s/w:
  - For example, **finding variables initialized but never used**.

A complete example

```
void stat_fun(int numbers[],int length)
{
  double med, var, sd, mean, sum, varsum;
  sum=0.0;
  for(i=0;i<length;i++)
     sum=sum+numbers[i];
  med=numbers[length/2];
  mean=sum/length;
  varsum=0.0;
  for(i=0;i<length;i++)
      varsum=varsum+(number[i]-mean)
         *(number[i]-mean);
  var=varsum/(length-1);
  sd=sqrt(var);
  printf("Length=%d",length);
  printf("Mean=%d",mean);
  printf("Median=%d",med);
  printf("Variance=%d",var);
  printf("Std. dev=%d",sd);
}
```

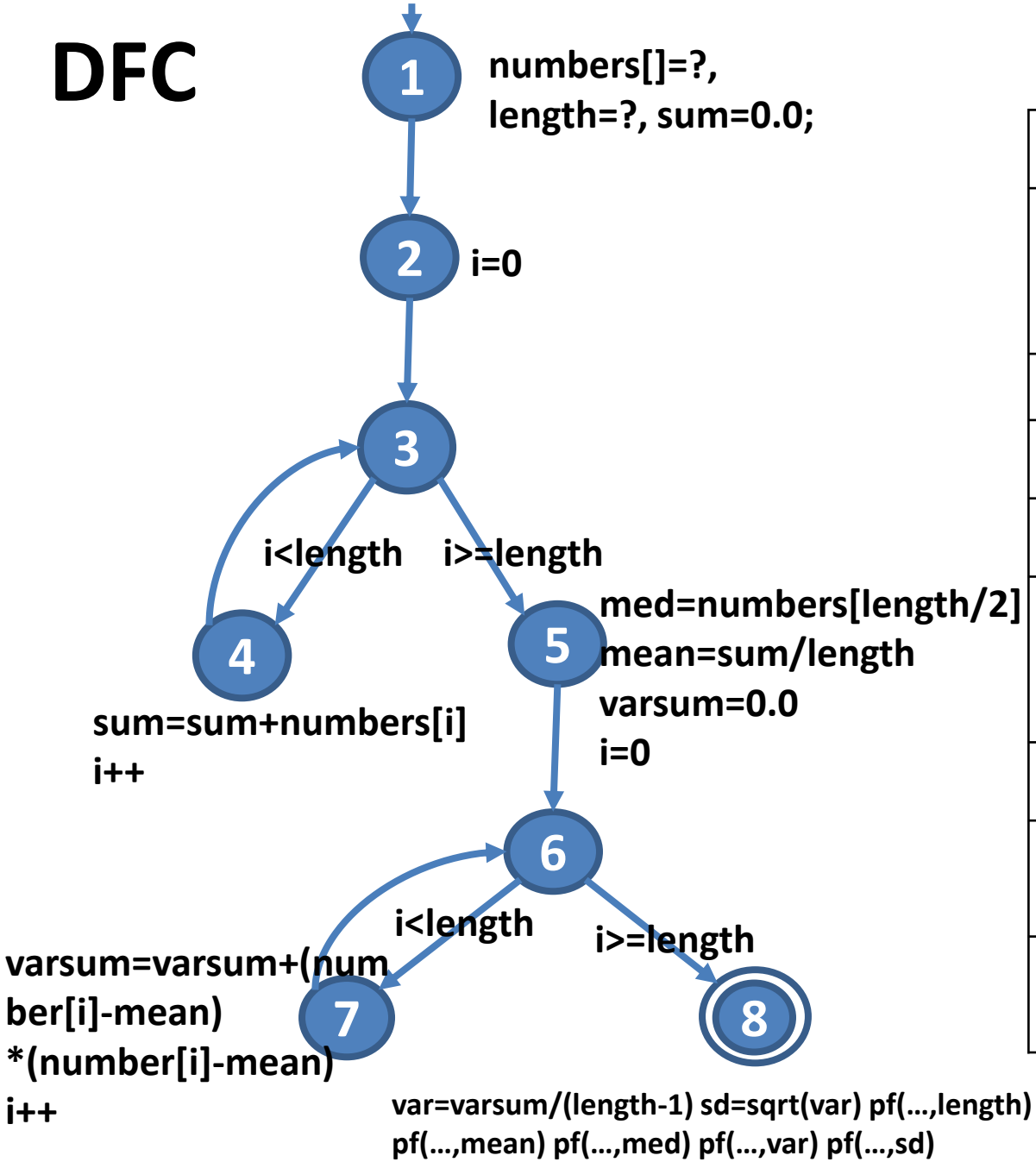


DFC



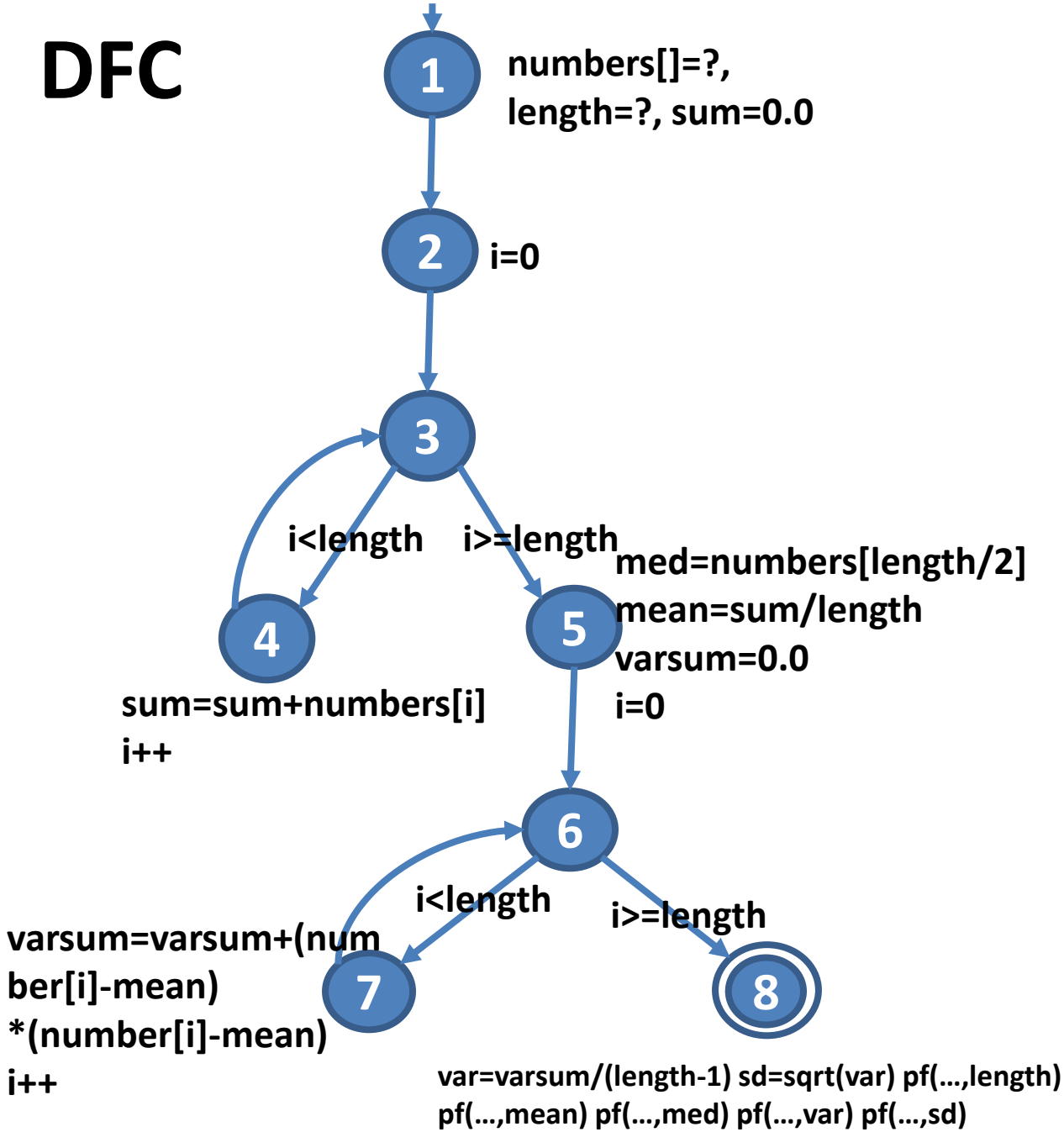
Node	Def	Use
1		
2		
3		
4		
5		
6		
7		
8		

DFC



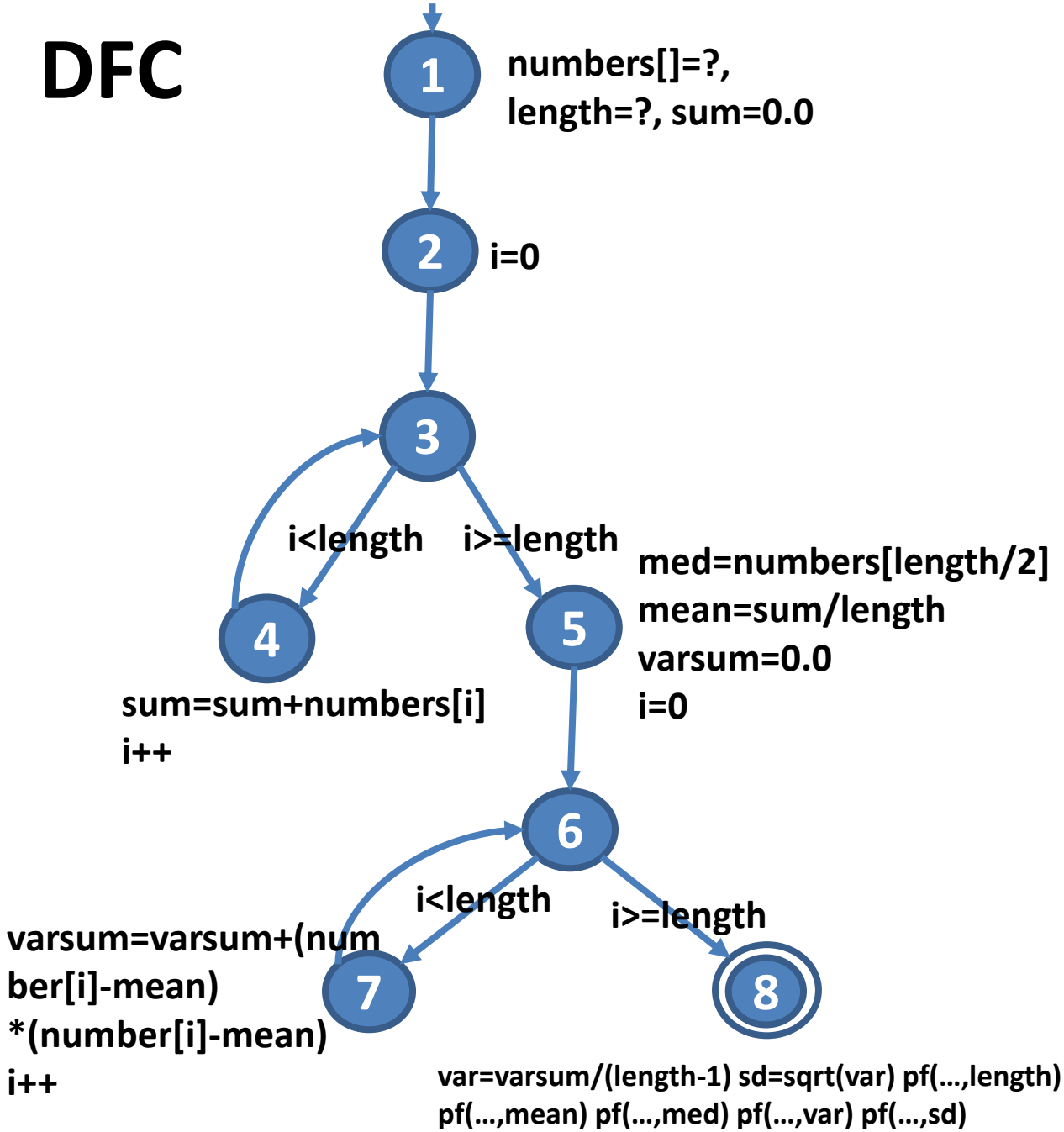
Node	Def	Use
1	{numbers, length, sum}	
2	{i}	
3		
4	{sum, i}	{numbers, sum, i}
5	{i,med, mean, varsum}	{numbers, length, sum}
6		
7	{i, varsum}	{numbers, mean, i, varsum}
8	{var, sd}	{length, med, var, sd, mean, varsum}

DFC


1
numbers[]=?,
length=?, sum=0.0
2
i=0
3
i<length
i>=length
4
sum=sum+numbers[i]
i++
5
med=numbers[length/2]
mean=sum/length
varsum=0.0
i=0
6
i<length
i>=length
7
varsum=varsum+(number[i]-mean)
*(number[i]-mean)
i++
8
var=varsum/(length-1) sd=sqrt(var) pf(…,length)
pf(…,mean) pf(…,med) pf(…,var) pf(…,sd)

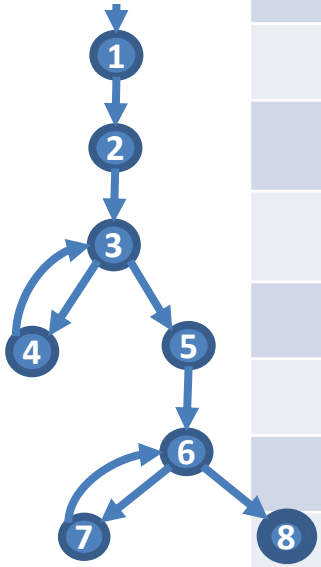
Edge	Use

DFC



Edge	Use
1,2	
2,3	
3,4	{i, length}
4,3	
3,5	{i, length}
5,6	
6,7	{i, length}
7,6	
6,8	{i, length}

DFC



Node	Def	Use
1	{numbers, length, sum}	
2	{i}	
3		
4	{sum, i}	{numbers, sum, i}
5	{i,med, mean, varsum}	{numbers, length, sum}
6		
7	{i, varsum}	{numbers, mean, i, varsum}
8	{var, sd}	{length,med,var,sd,mean,varsum}

Edge	Use
1,2	
2,3	
3,4	{i, length}
4,3	
3,5	{i, length}
5,6	
6,7	{i, length}
7,6	
6,8	{i, length}

Variables	du pairs
numbers	(1,4) (1,5) (1,7)
length	(1,5) (1,8) (1,(3,4)) (1,(3,5)) (1,(6,7)) (1,(6,8))
med	(5,8)
var	(8,8)
sd	(8,8)
mean	(5,7) (5,8)

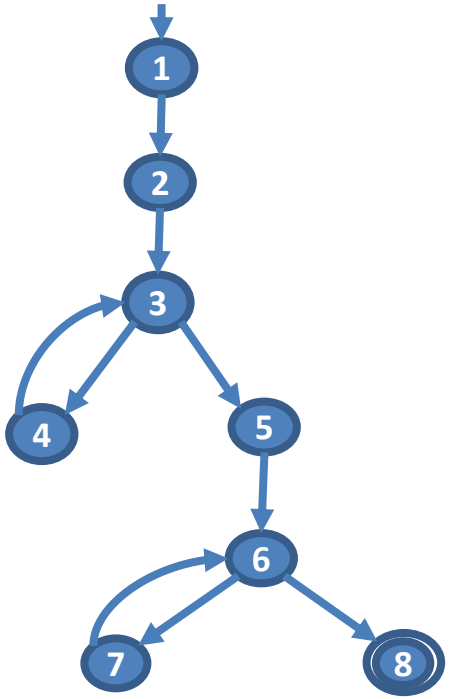
Variables	du pairs
sum	(1,4) (1,5) (4,4) (4,5)
varsum	(5,7 (5,8) (7,7) (7,8)
i ~~No paths~~	(2,4) (2,(3,4)) (2,(3,5)) (2,7) (2,(6,7)) (2,(6,8)) (4,4) (4,(3,4)) (4,(3,5)) (4,7) (4,(6,7)) (4,(6,8)) ~~(5,4) (5,34) (5,35)~~ (5,7) (5,(6,7)) (5,(6,8)) (7,7) (7,(6,7)) (7,(6,8))

DFC

Variables	du pairs	du paths
numbers	(1,4) (1,5) (1,7)	(1,2,3,4) (1,2,3,5) (1,2,3,5,6,7)
length	(1,5) (1,8) (1,34) (1,35) (1,67) (1,68)	(1,2,3,5) (1,2,3,5,6,8) (1,2,3,4) (1,2,3,5) (1,2,3,5,6,7) (1,2,3,5,6,8)
med	(5,8)	(5,6,8)
var	~~(8,8)~~	No path needed, ignore this as is not def clear
sd	~~(8,8)~~	No path needed ignore this as is not def clear
mean	(5,7) (5,8)	(5,6,7) (5,6,8)
sum	(1,4) (1,5) (4,4) (4,5)	(1,2,3,4) (1,2,3,5) (4,3,4) (4,3,5)
varsum	(5,7 (5,8) (7,7) (7,8)	(5,6,7) (5,6,8) (7,6,7) (7,6,7)
i	(2,4) (2,34) (2,35) ~~(2,7) (2,67) (2,68)~~ (4,4) (4,34) (4,35) ~~(4,7) (4,67) (4,68)~~ (5,7) (5,67) (5,68) (7,7) (7,67) (7,68) ignore these as are not def clear	(2,3,4) (2,3,4) (2,3,5) (4,3,4) (4,3,4) (4,3,5) (5,6,7) (5,6,7) (5,6,8) (7,6,7) (7,6,7) (7,6,8)

DFC

Variables	UNIQUE (du paths, delete duplicate du paths)
numbers	(1,2,3,4) (1,2,3,5) (1,2,3,5,6,7)
length	~~(1,2,3,5)~~ (1,2,3,5,6,8) ~~(1,2,3,4)~~ ~~(1,2,3,5)~~ ~~(1,2,3,5,6,7)~~ ~~(1,2,3,5,6,8)~~
med	(5,6,8)
var	
sd	
mean	(5,6,7) ~~(5,6,8)~~
sum	~~(1,2,3,4)~~ ~~(1,2,3,5)~~ (4,3,4) (4,3,5)
varsum	~~(5,6,7)~~ ~~(5,6,8)~~ (7,6,7) ~~(7,6,7)~~
i	(2,3,4) ~~(2,3,4)~~ (2,3,5) ~~(4,3,4)~~ ~~(4,3,4)~~ ~~(4,3,5)~~ ~~(5,6,7)~~ ~~(5,6,7)~~ ~~(5,6,8)~~ ~~(7,6,7)~~ ~~(7,6,7)~~ (7,6,8)



Paths those skip a loop **(1,2,3,5) (1,2,3,5,6,8) (2,3,5) (5,6,8)**

Paths with one iterations of a loop **(1,2,3,4) (1,2,3,5,6,7) (5,6,7) (4,3,5) (2,3,4) (7,6,8)**

Paths with 2 iterations of a loop **(4,3,4) (7,6,7)**

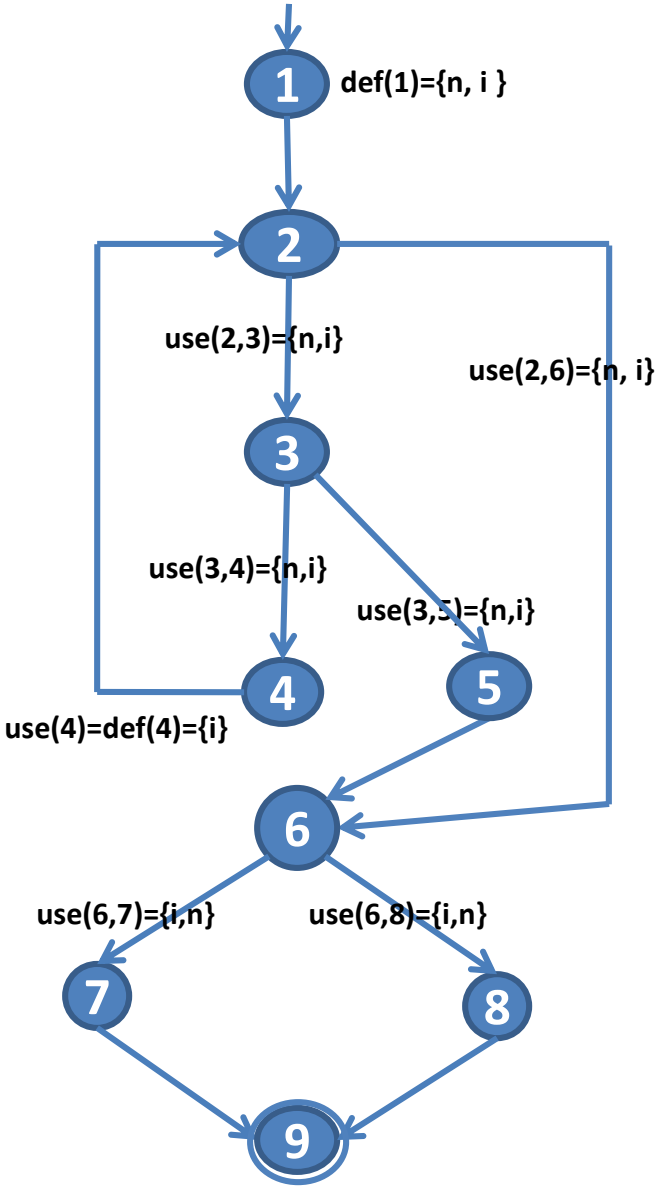
DFC

Paths those skip a loop	(1,2,3,5) (1,2,3,5,6,8) (2,3,5) (5,6,8)
Paths with one iterations of a loop	(1,2,3,4) (1,2,3,5,6,7) (5,6,7) (4,3,5) (2,3,4) (7,6,8)
Paths with 2 iterations of a loop	(4,3,4) (7,6,7)

Paths those skip a loop	**Test case: Array contain NO value, length =0** **Test path: Execution fails, A bug found**
Paths with one iterations of a loop	**Test case: Array contain one value, length =1** **Test path: 1,2,3,4,3,5,6,7,6,8**
Paths with 2 iterations of a loop	**Test case: Array contain more than one value, length =3** **Test path: 1,2,3,4,3,4,3,4,3,5,6,7,6,7,6,7,6,8**

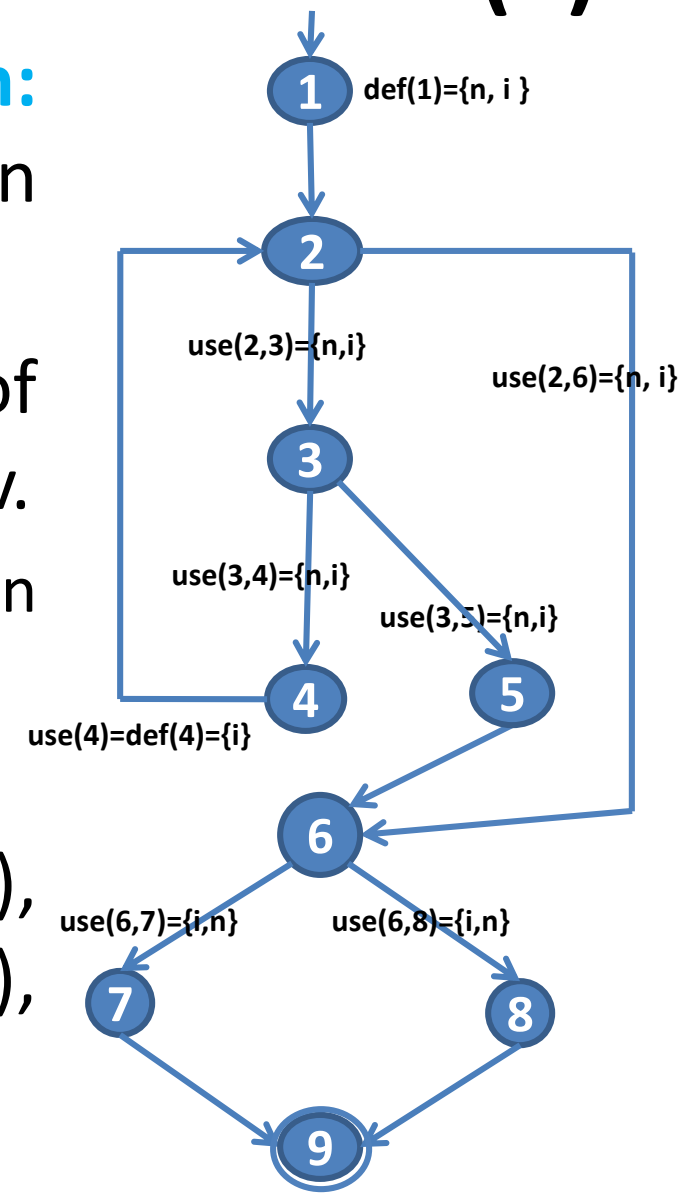
Data flow coverage criteria

- **Data flow coverage criteria** is defined as **a set of du-paths**.
- **du-paths** are **grouped** to define the data flow coverage criteria
  1. **Grouping** as per **definition**
  2. **Grouping** as **per definition and use**
- **Data flow coverage criteria checks for definition of variables reaching their uses in different ways**.



Grouping du-paths: as per definition (1)

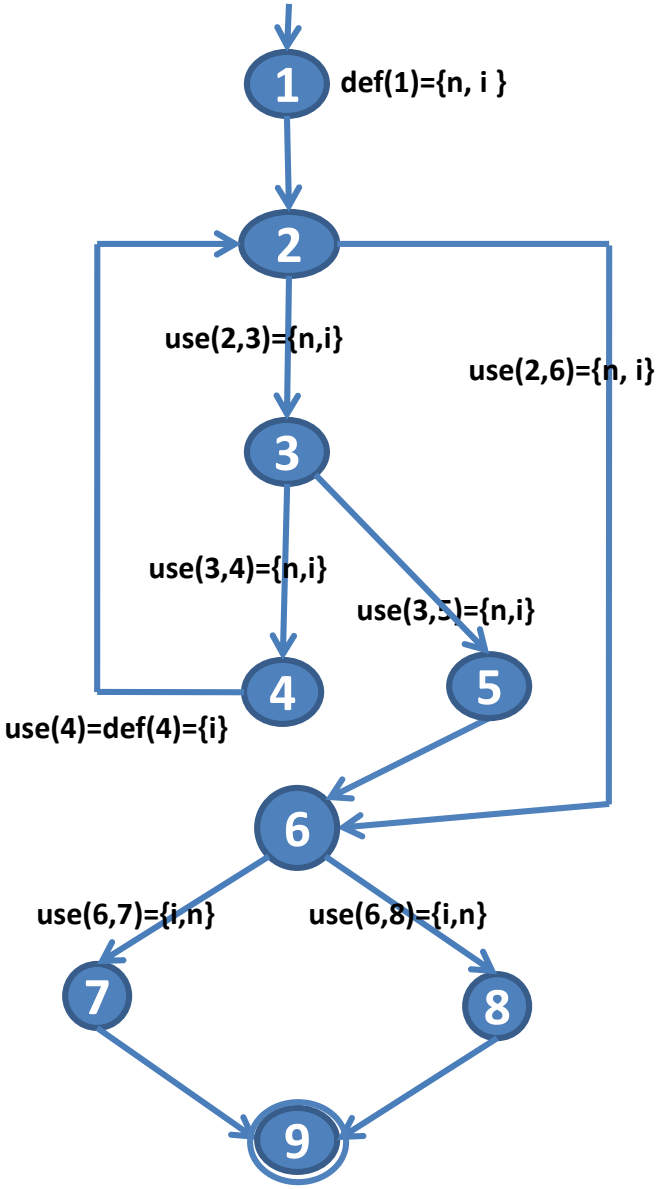
- **Grouping according to definition:** Consider all du-paths w.r.t. a given variables v that start at a node.
- **The def path set** du(n_i, v): The set of du-paths that starts at n_i for variable v.
  - For a large program, the def-path set can be large.
  - du-paths are not grouped by uses.
- du(4,i)={(4,4), (4,2), (4,2,3), (4,2,3,4), (4,2,3,5), (4,2,3,5,6,7), (4,2,3,5,6,8), (4,2,6), (4,2,6,7), (4,2,6,8)}
- du(1,i)={?}, du(1,n)={?}



Can we have the path (4,4) in du(4,i)?

Grouping du-paths: as per definition & use (2)

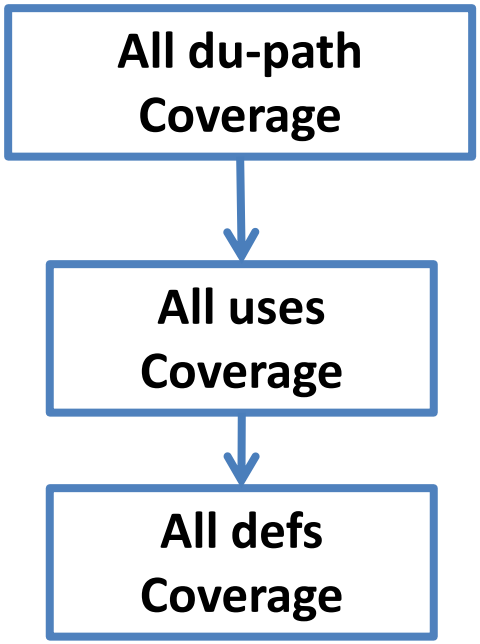
- **Grouping according to definition and use** allows definitions to flow to use.
- **A def-pair set,** du(n_i, n_j, v) is the set of du-paths from n_i to n_j for variable v.
  - du(4,4,i)={(4,4), (4,2,3,4)}
  - du(4,2,i)={(4,2)}
  - du(4,3,i)={(4,2,3)}
  - du(4,5,i)={(4,2,3,5)}
  - du(4,6,i)={(4,2,6), (4,2,3,5,6)}
  - du(4,7,i)={(4,2,6,7), (4,2,3,5, 6,7)}
  - du(4,8,i)={(4,2,6,8), (4,2,3,5,6,8)}
- A def-pair set collects together all the simple ways to get from a given definition to a given use.
- **A def-pair set for a def at node n_i is the union of all the def-path sets for that def.**
  - du(ni,v) = U_{nj} du(n_i, n_j, v)



Data Flow coverage criteria

- The **common Data flow criteria** are
  - TR: **Each of the def reaches at least one use**
    - For each def-path set S=du(n, v), TR contains at least one path d in S
    - **All def coverage**
  - TR: **Each of the def reaches all possible uses**
    - For each def-pair set S=du(n_i, n_j, v), TR contains at least one path d in S
    - **All uses coverage**
  - TR: **Each of the def reaches all possible uses through all possible du-paths.**
    - For each def-pair set S= du(n_i,n_j,v), TR contains every path d in S
    - **All du-path coverage**

Data Flow Coverage Criteria

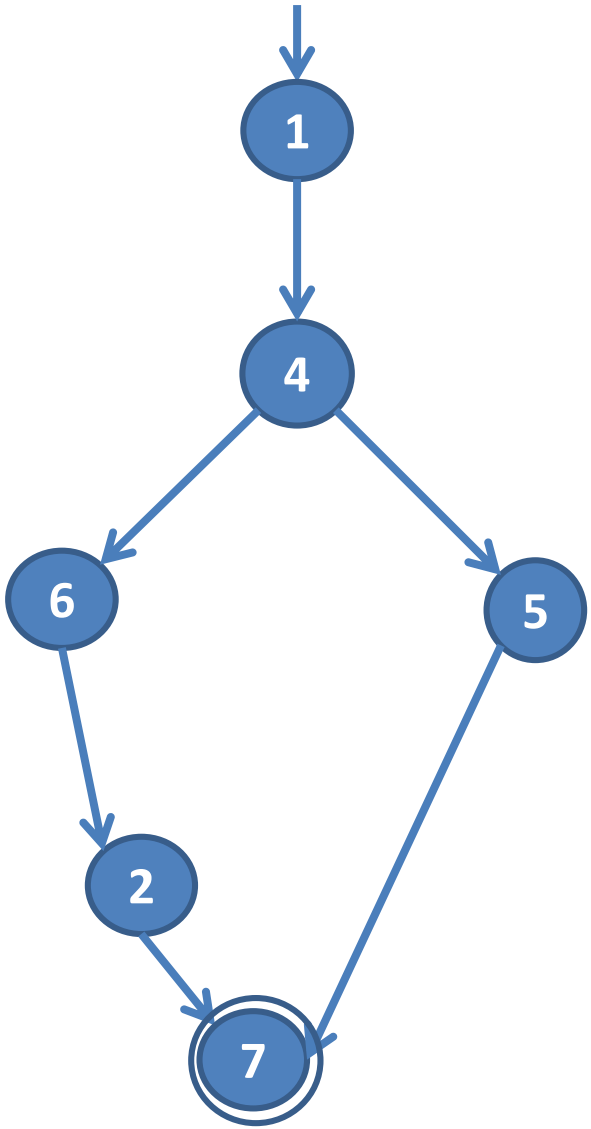


Traditional Terminologies used in White box testing

- **Code Coverage:** Statement coverage, branch coverage, decision coverage, modified condition decision coverage (MCDC), path coverage etc.
  - **Node coverage** is same as **statement coverage**
  - **Edge coverage** is same as **branch coverage**
  - **Prime path coverage** is same as **loop coverage**
  - **Decision coverage** and MCDC are related to logical predicates
- **Cyclomatic complexity:** Basis path testing, structural testing
- **Data flow testing:** Data flow coverage
- Decision to decision path (**DD-path**) coverage

Decision to decision paths

- A **DD path** is a **path of execution between two decisions** in the CFG
- Here the **TR** **is to cover** **all paths** **between two decision points**.
- **Nodes** **where multiple edges merge** are considered as a **decision points**.
- In this example, **all the paths between node 4 and node 7** in the example are to be covered.



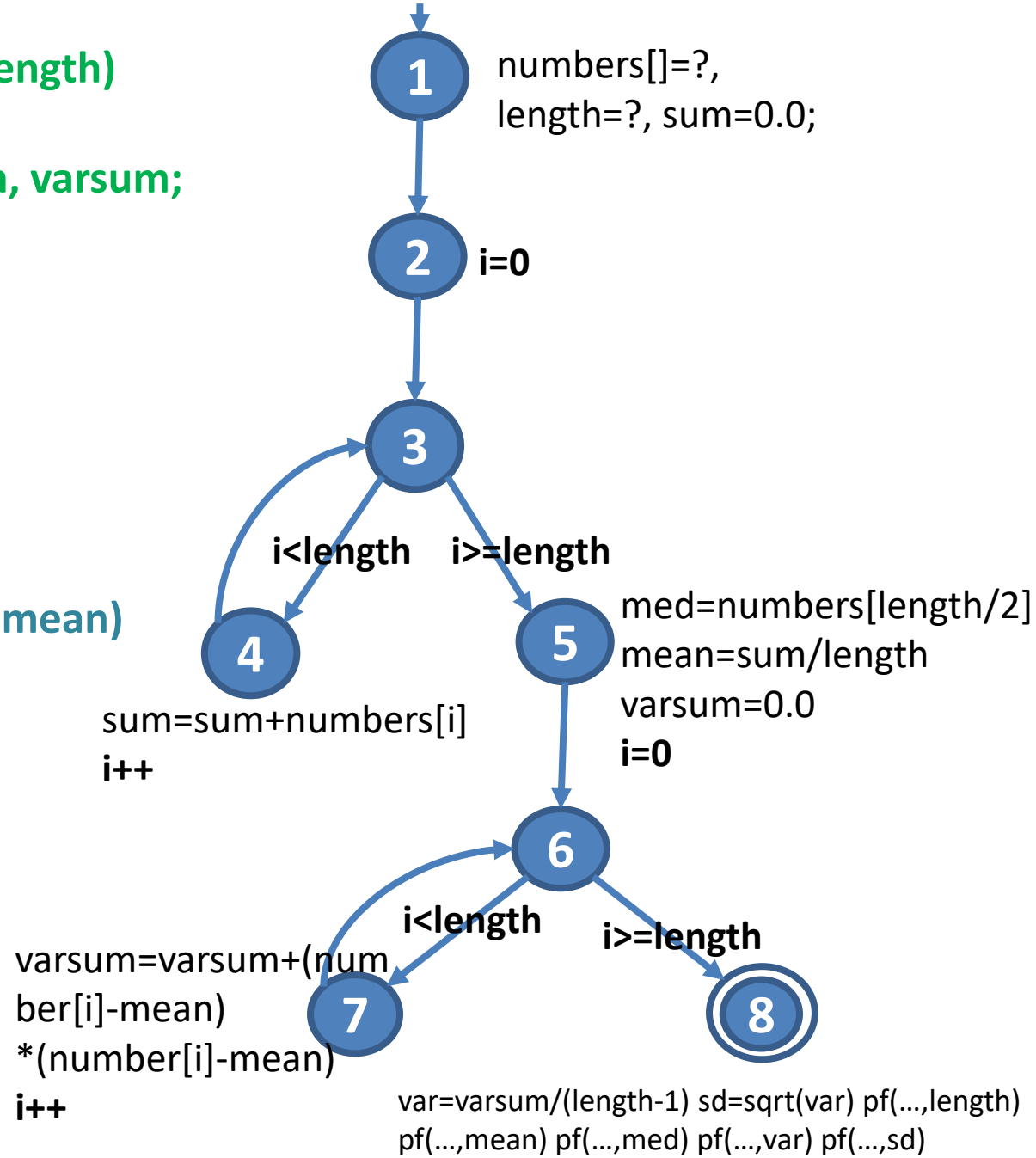
Basis path testing

- **Basis path testing** is to test each **linearly independent path** in the CFG
  - A linearly independent path in a CFG is a path that does not contain other paths within it. **Similar to prime path** (except prime path is a simple path)

Cyclomatic complexity

Control flow graph (CFG) can be utilized to **estimate** **how difficult** a program is to **understand** for a **software engineer** and can **help estimating time** and **effort** required for **testing**.

```
void stat_fun(int numbers[],int length)
{
  double med, var, sd, mean, sum, varsum;
  sum=0.0;
  for(i=0;i<length;i++)
     sum=sum+numbers[i];
  med=numbers[length/2];
  mean=sum/length;
  varsum=0.0;
  for(i=0;i<length;i++)
      varsum=varsum+(number[i]-mean)
         *(number[i]-mean);
  var=varsum/(length-1);
  sd=sqrt(var);
  printf("Length=%d",length);
  printf("Mean=%d",mean);
  printf("Median=%d",med);
  printf("Variance=%d",var);
  printf("Std. dev=%d",sd);
}
```



Cyclomatic complexity (McCabe 1972)

- **Cyclomatic complexity** is a software metric used to indicate the **structural complexity** of a program
- Cyclomatic complexity represents the **number of linearly independent paths** in the **control flow graph of a program**.
- A **linearly independent path** is **any path through** the **program** that introduces **at least one new edge** that is **not included** in **any other linearly independent paths**.
- For a **complicated program** it is **not easy to determine** the **number of linearly independent paths** of the program.

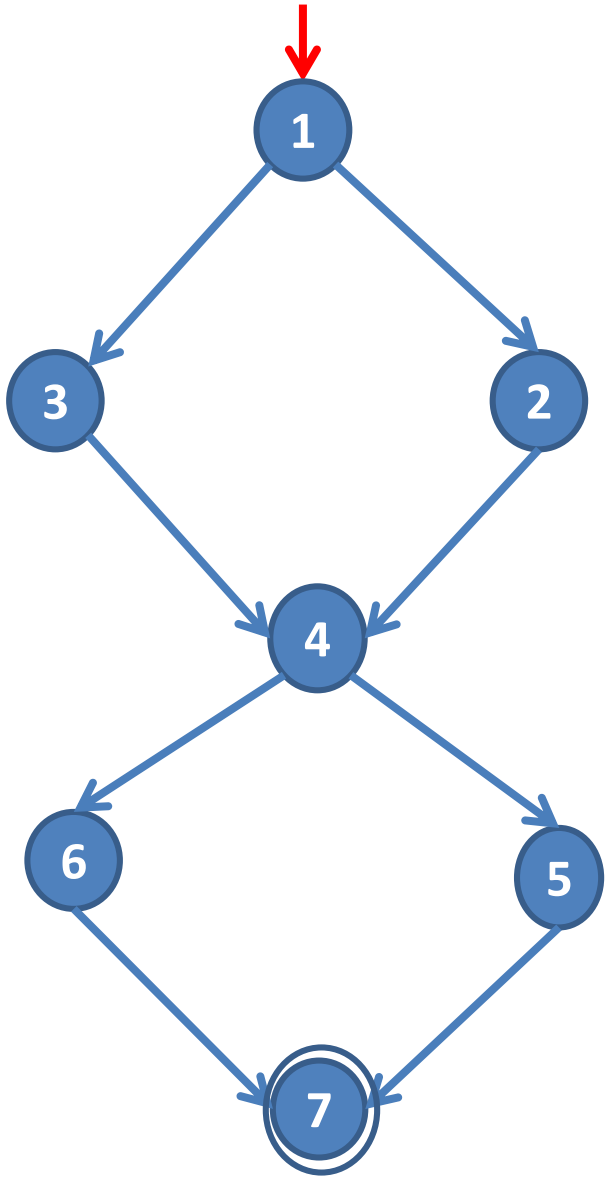
Cyclomatic complexity (McCabe 1972)

- **McCabe's cyclomatic complexity metric provides a practical way of estimating the maximum number of linearly independent paths** in a program.
- **McCabe's cyclomatic complexity defines an upper bound** for the **number of linearly independent paths** through a program.
- Also, the **McCabe's cyclomatic complexity** is **very simple** to compute.
- Though the McCabe's metric does **not directly identify the linearly independent paths**, but **it informs approximately how many paths** to look for in a program.
- There are **three different ways to compute the cyclomatic complexity**.

Compute Cyclomatic complexity

- **Method 1**:
  - Given a control flow graph G of a program, the cyclomatic complexity V(G) can be computed as:

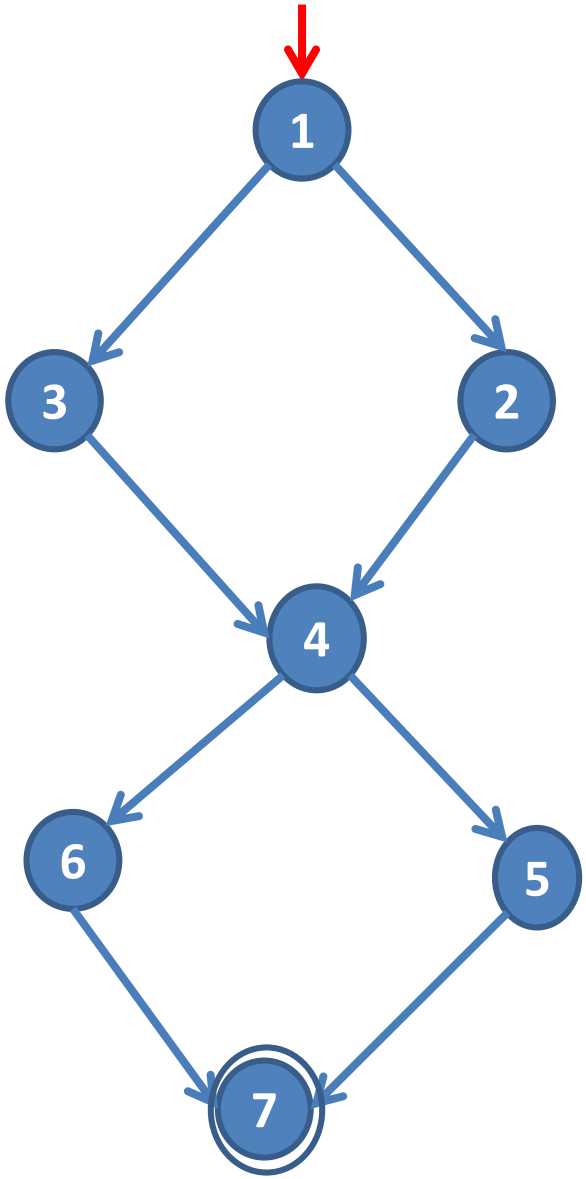
    V(G) = E – N + 2
  - where **N** is the **number of nodes** of the control flow graph and **E** is the **number of edges** in the control flow graph.
  - For the CFG, E=8 and N=7. Therefore, the Cyclomatic Complexity = 8-7+2 = 3.



Compute Cyclomatic complexity

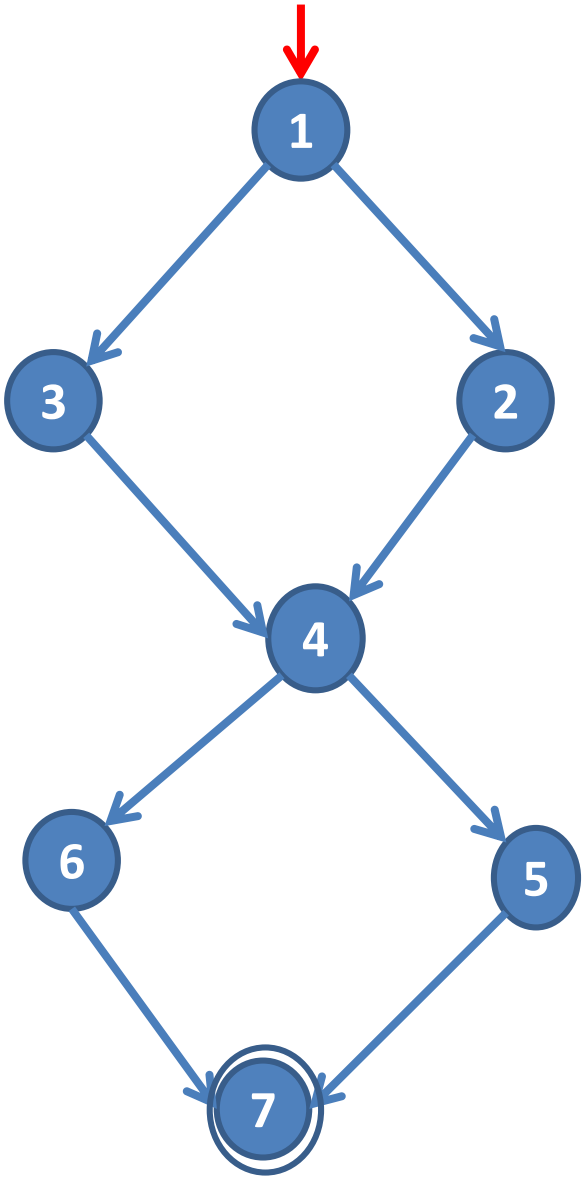
- **Method 2**:
  - **Cyclomatic complexity** of a program can be computed as:

    V(G) = Total number of bounded areas + 1
  - In the program's control flow graph G, any **region enclosed by nodes and edges** can be called as a **bounded area**. This is an easy way to determine the McCabe's cyclomatic complexity.
  - For this CFG, no. of bounded area is 2. Therefore, the cyclomatic complexity = 3.
  - But, what if the graph G is not planar (i.e. when the graph is drawn, two or more edges intersect)?
    - Actually, it can be shown that **structured programs always yield planar graphs**. But, **presence of GOTO's can easily add intersecting edges**. Therefore, for non-structured programs, this way of computing the McCabe's cyclomatic complexity cannot be used.



Compute Cyclomatic complexity

- **Method 3**:
  - The cyclomatic complexity of a program can also be easily computed by computing the number of decision statements of the program.
  - If **N** is the **number of decision statement** of a program, then the **McCabe's metric is** equal to **N+1**
  - For the CFG of prime number program example, N=2. Therefore, the cyclomatic complexity = 2+1 = 3.



Other Applications of McCabe's Cyclomatic Complexity

- **Cyclomatic complexity** is higher for a program containing many decision statements and therefore **can be an estimate of**
  - **Structural Complexity**
  - **Testing Effort**
  - **Program Readability**
- If cyclomatic complexity of a program is very high then structural complexity of the program is very high and accordingly more testing effort is required for the program and the readability of the program is also poor.
- If CC is 1, the program is a sequence of statements and may be a very simple one.
- A **software module** is expected to have **CC between 2 to 10**.